B Trees

- The idea is, store as many Keys into a node such that, a node perfectly fits into a block of HDD.
- The larger the number of Keys in a node (high value of m), the minimum the height of the tree.
- All Keys in a node are stored in a sorted fashion. And since Keys are sorted in a node, we can perform binary search there.

- A B tree is a M-way search tree with rules.

Rules :-

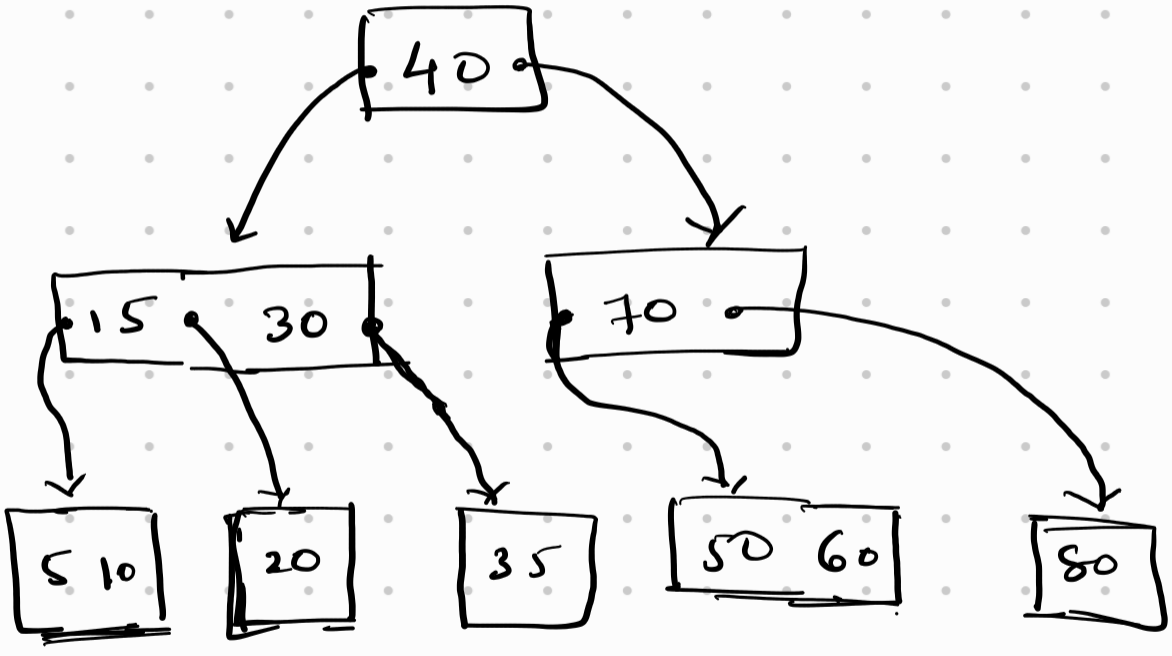
1) $\lceil \frac{M}{2} \rceil$ children ; $\lceil \frac{M}{2} \rceil - 1$ Keys per node at least.
2) The above rule is not applicable on the root.
3) All leaf nodes must be at same level.
4) Creation process is bottom to up. Elements are inserted to leaf nodes.

A B-tree node that is used for indexing has :-

1) Keys
2) Child pointers
3) Record pointers (To point to blocks where actual data is stored)

10, 20, 40, 50, 60, 70, 80, 30, 35, 15, 5

m = 4 Keys = 3



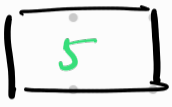
Insert :- 5, 3, 21, 9, 13, 22, 7, 11, 14, 8, 16

Insert :- 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16

order (m) = 4

- Min. Keys each node can have :- $\left\lceil \frac{4}{2} \right\rceil - 1 = 1$
- Min children allowed = $\left\lceil \frac{4}{2} \right\rceil = 2$
- Max Keys allowed = $m - 1 = 4 - 1 = 3$

Inserting 5 :



Inserting 3 :

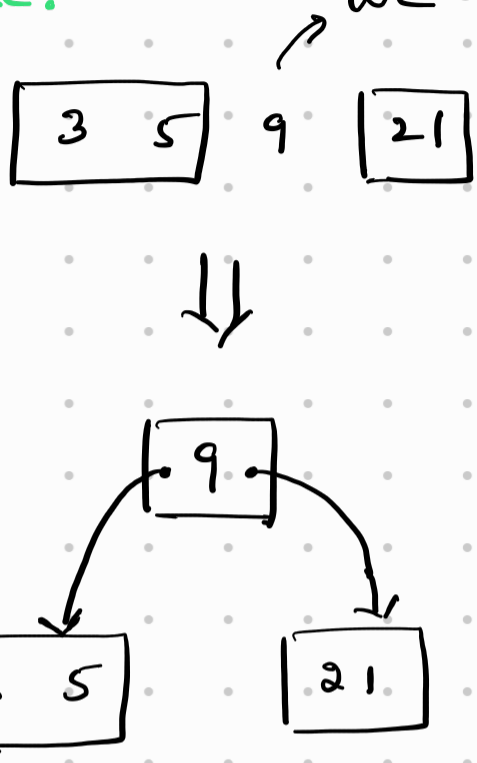
Inserting 21 :

Inserting 9 :

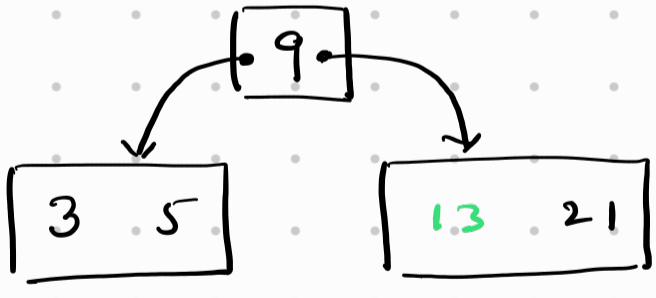
Now, our Keys have exceeded the max limit, so, we split the node & move the median upwards. Median of 3, 5, 9, 21 can be 5 or 9. It's our choice to be left biased or right biased.

we will be left biased. So, left nodes will have more Keys.

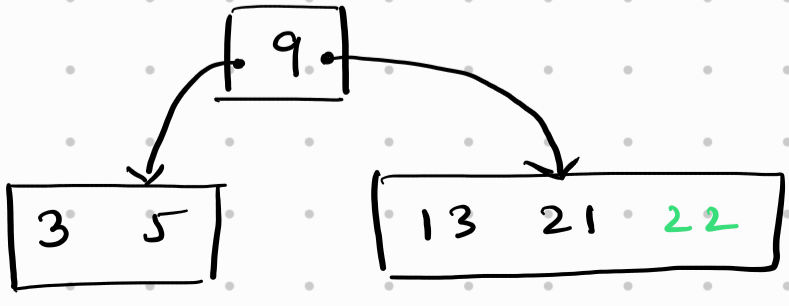
Splitting node:



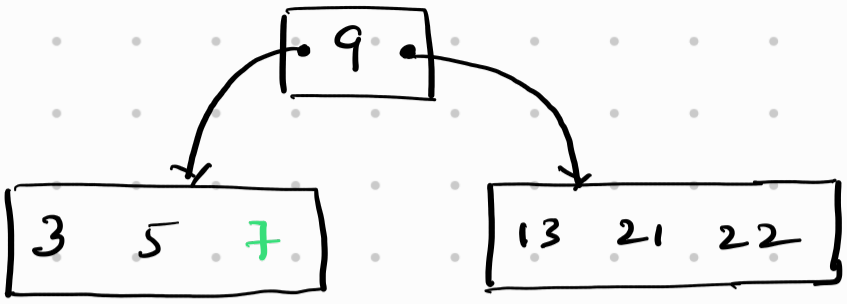
Insert 13:



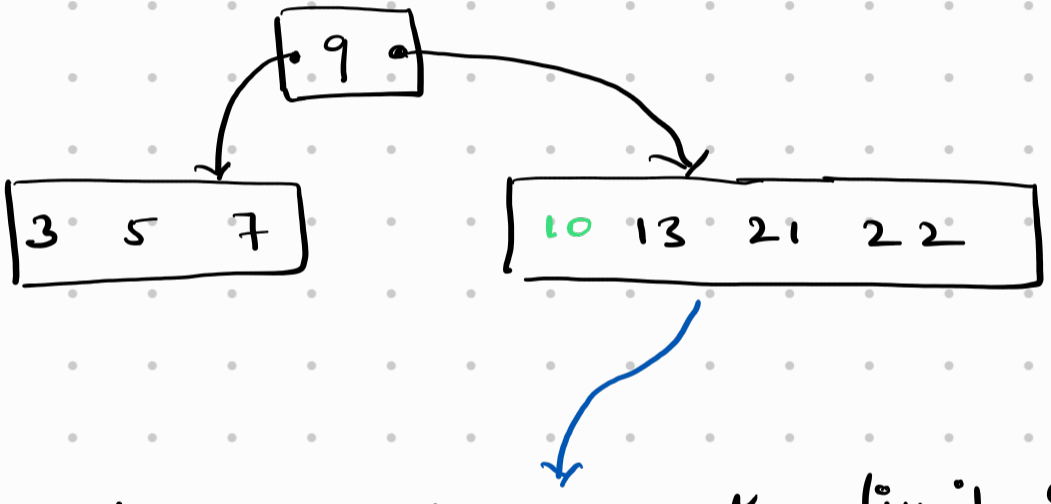
Insert 22:



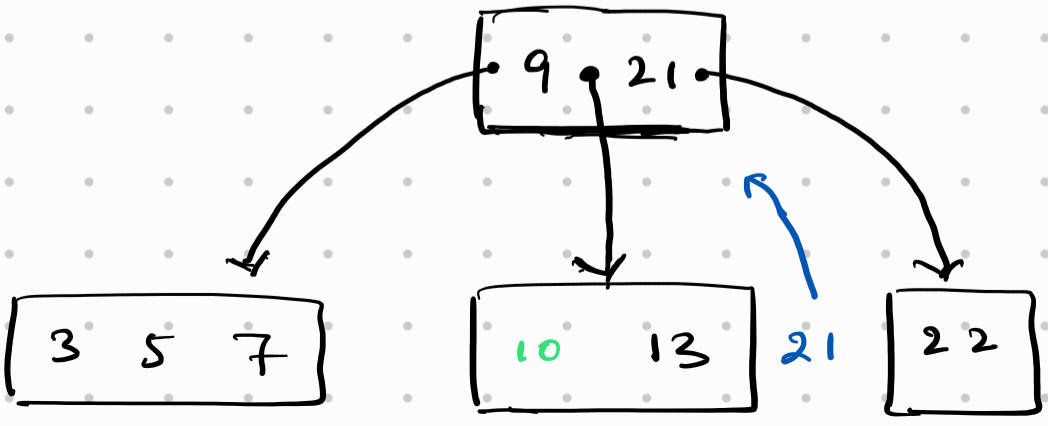
Insert 7 :



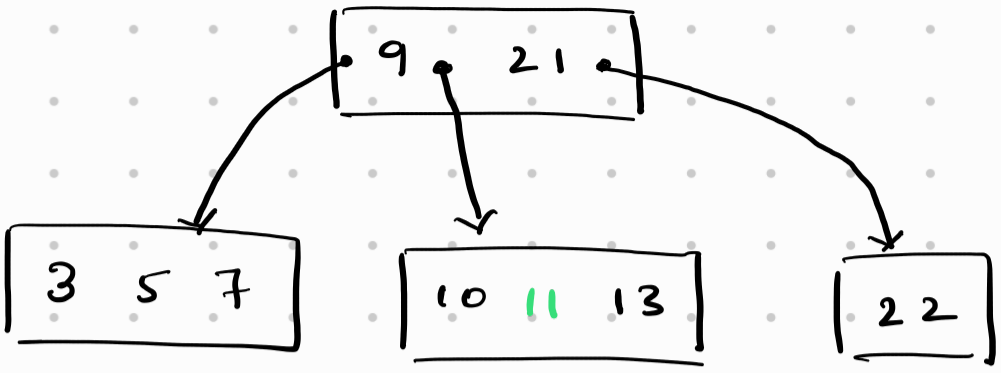
Insert 10 :



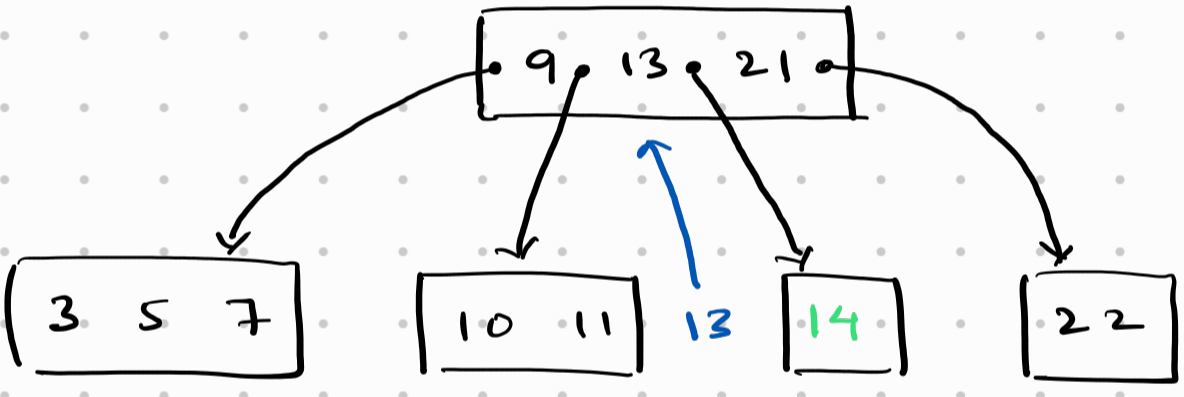
Has exceeded the max Key limit. Split.



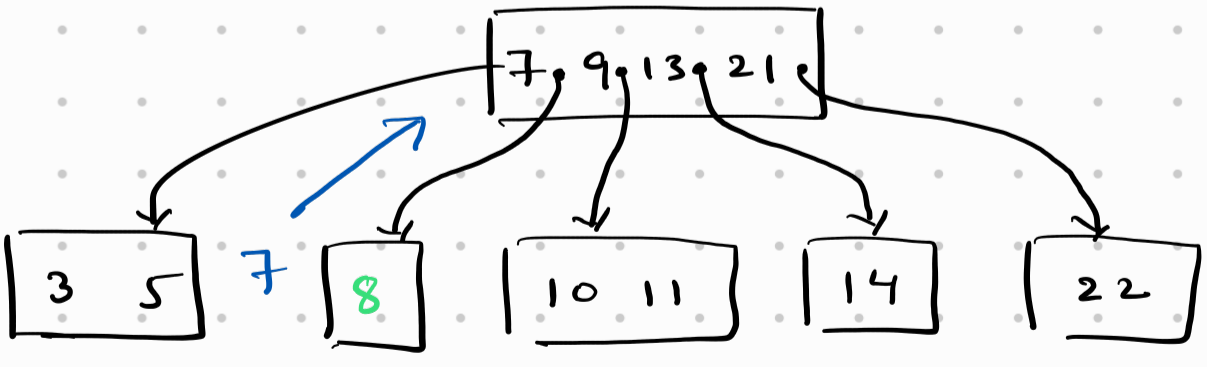
Insert 11 :



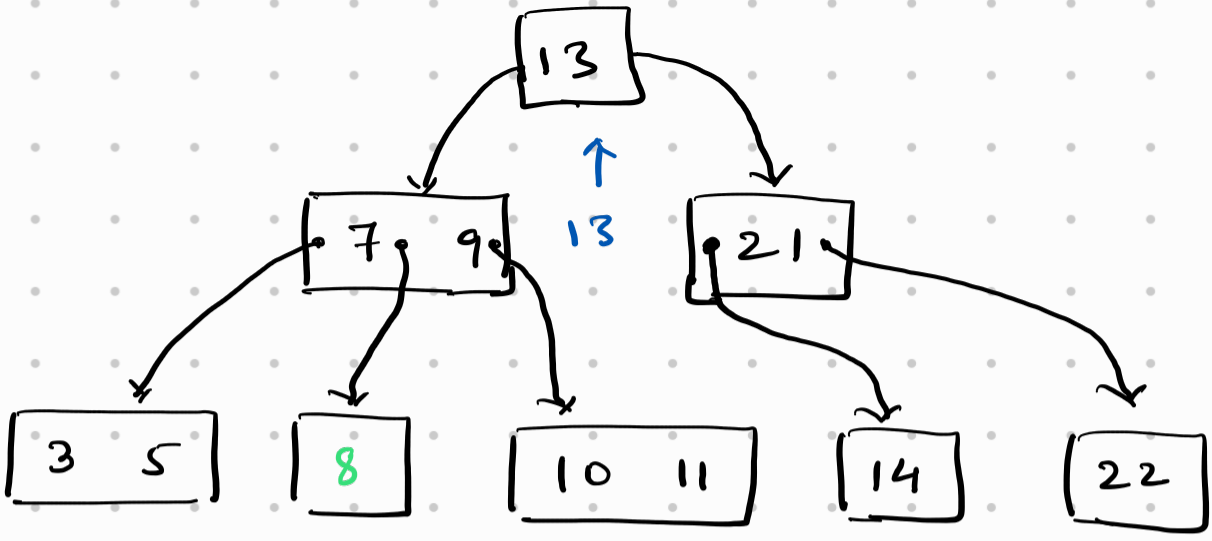
Insert 14 :



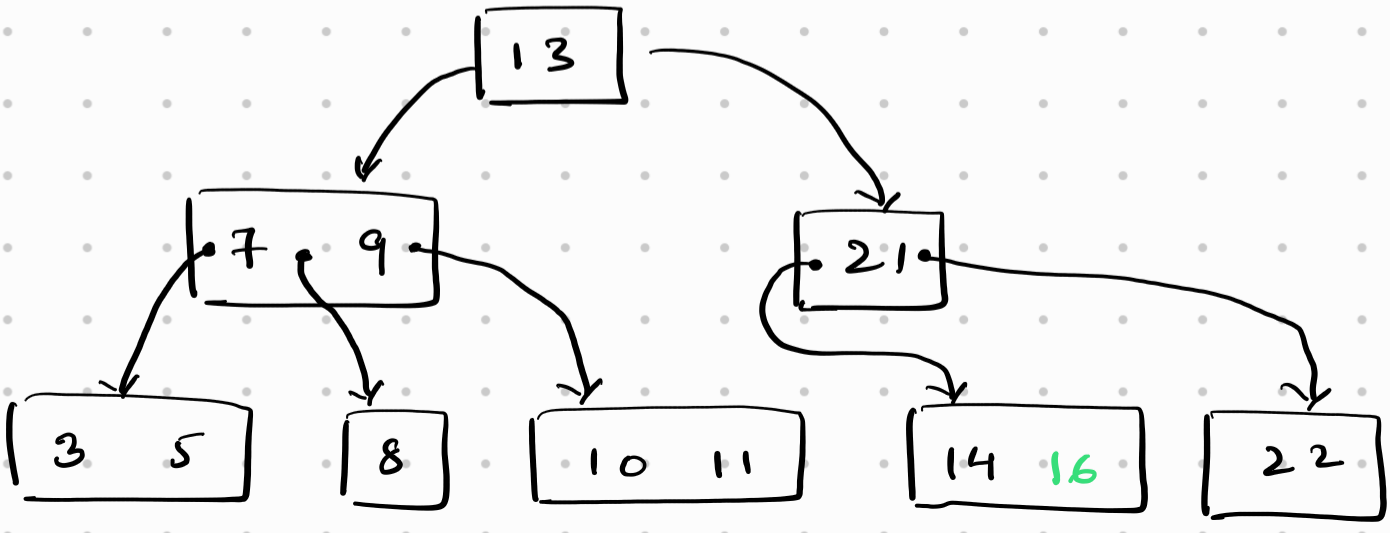
Insert 8 :



But, here when we sent 7 to parent node, the parent node also overflowed. So, we split parent node as well :-



Insert 16 :-



Deletion in B-Trees :-

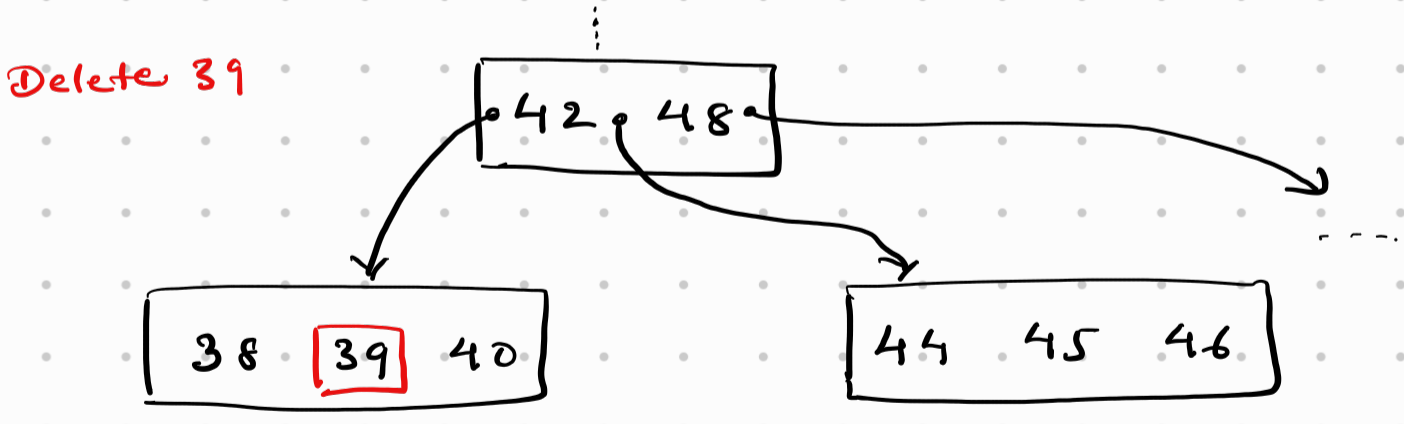
Case 1 :- The Key to be deleted is in a leaf node.

Case 2 :- The Key to be deleted is in an internal node.

Case 1 : Scenario 1

The leaf node in which the Key lies has more than minimum no. of Keys allowed. Just delete the Key.

For eg :- $m = 5$, minimum Key $= \lceil \frac{5}{2} \rceil - 1 = 2$

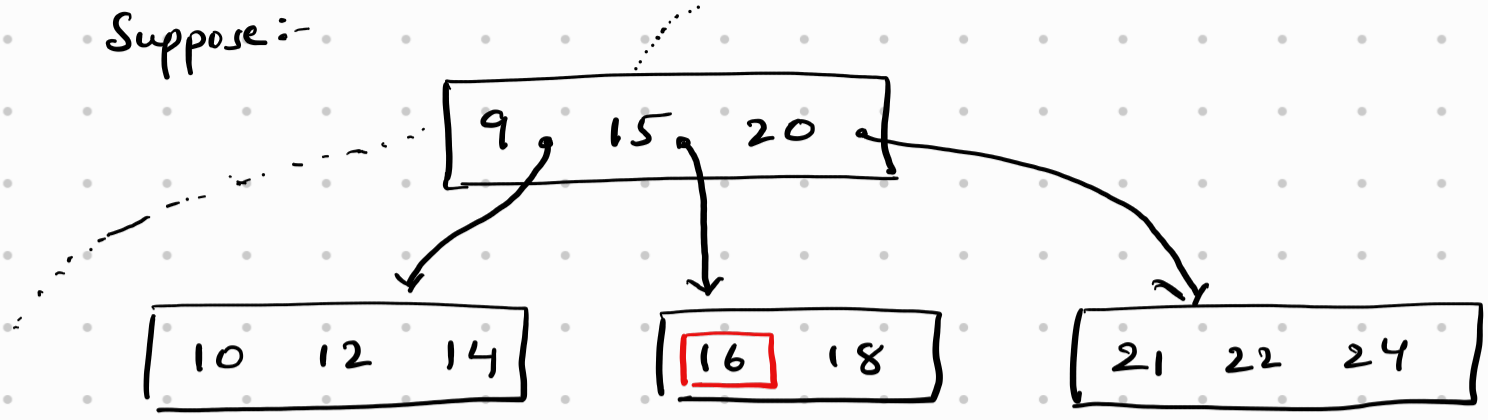


Here, deleting 39, will leave the node with 2 Keys. which is fine. We can have minimum 2 Keys. That's allowed.

Case 1 : Scenario 2

Delete 16

Suppose :-



If we delete 16, the node would be left with just 1 node, which violates the minimum Key Condition of 2.

So, what do we do?

→ we see that the sibling nodes have more than minimum nodes.

Left node :- 10 12 14

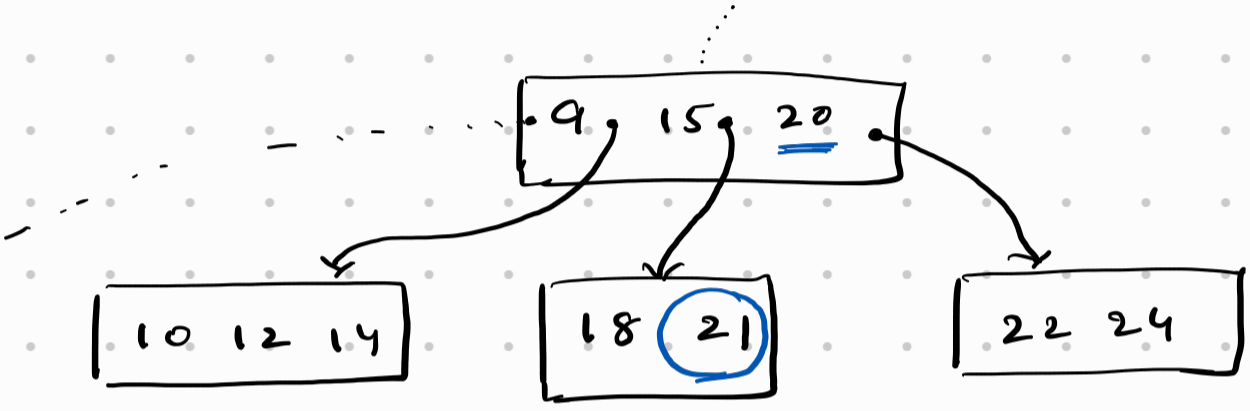
Right sibling node :- 21 22 24

Both of them have more than minimum keys. we can decide to borrow from any one of them.

→ If we decide to take from left node, we have to take the greatest key 14

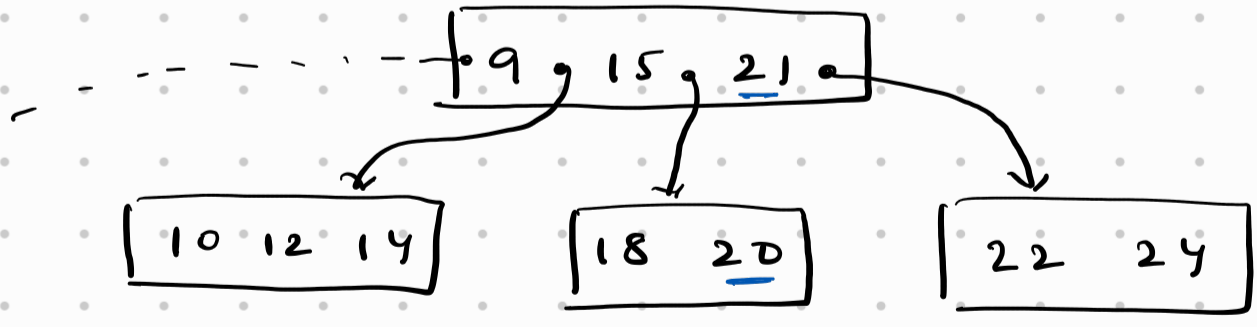
→ If we decide to take from right node, we have to take the smallest Key 21.

Let's suppose we decide to take from right node. We take 21. Now, the status would be :-



Wait a minute, now the separator 20 in the parent node is wrong. 21 is bigger than 20.

→ we can solve this by swapping 21 & 20.



Now, the separator is right.

→ This works as long as there is a sibling we can take from.

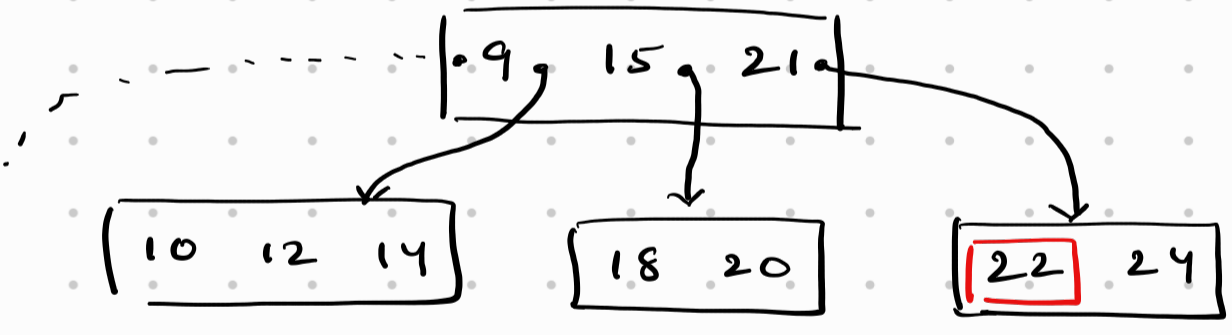
Case 1 : Scenario 3 :

What happens when we remove a key and our sibling nodes are also at their minimum? This time we can't take from sibling.

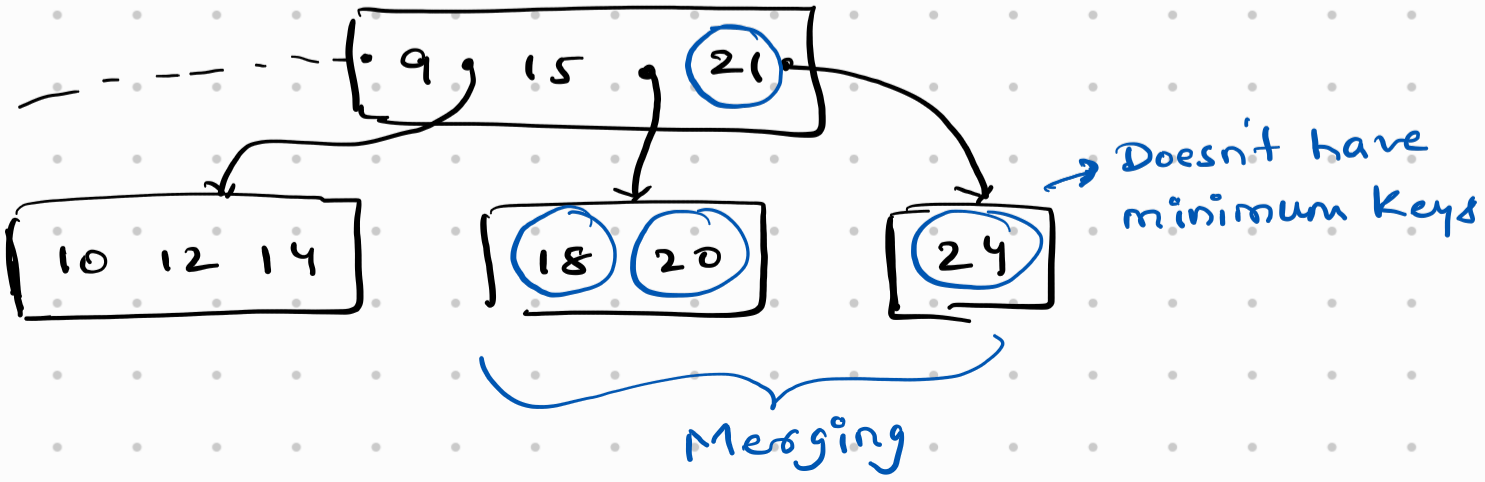
Just like while inserting keys we could split nodes apart, while deleting keys we might merge nodes back together.

→ Take the current node, one sibling node & the separator in the parent node and merge them together into a single full node.

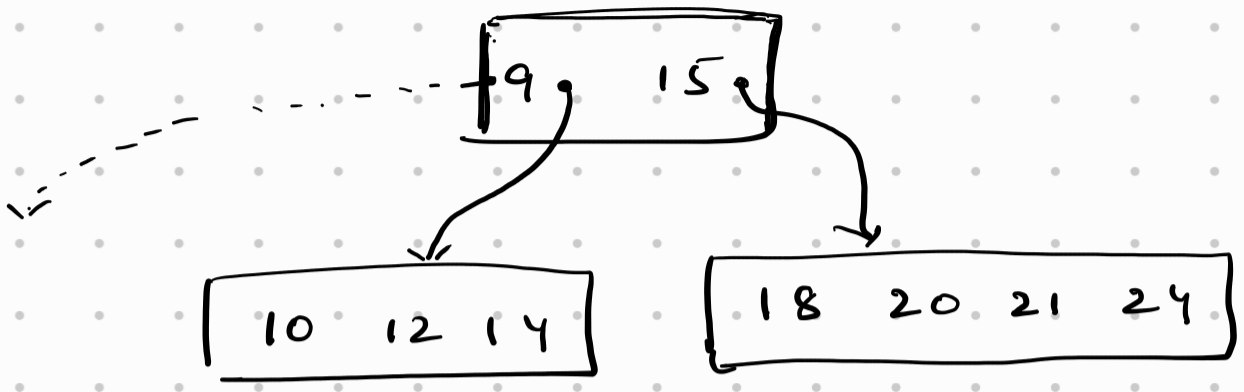
Delete 22



⇓ After deleting



After merging 18, 20, 21, 24



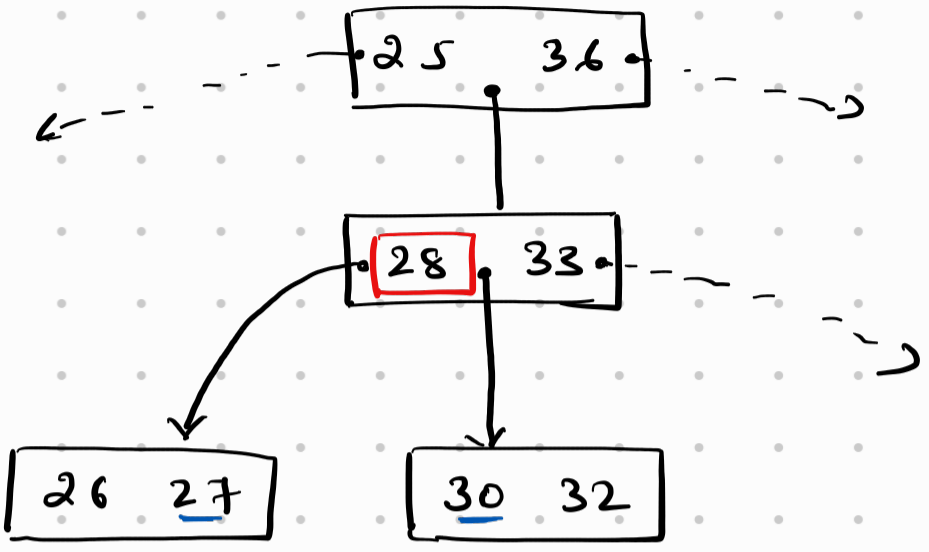
Note :- Why merging the two nodes will not overflow.

→ Left node as minimum keys.
→ right node has less than minimum.
→ one key from parent
→ Summing them up will always be less than the max allowed.. or max.

→ If by taking an element (separator) from the parent node leaves the node with less than minimum number of keys, we apply the same merging technique recursively upwards.

Case 2 : Deleting a key in internal node.

Delete 28



If we delete 28, well it is also acting as a separator for its left & right subtree.

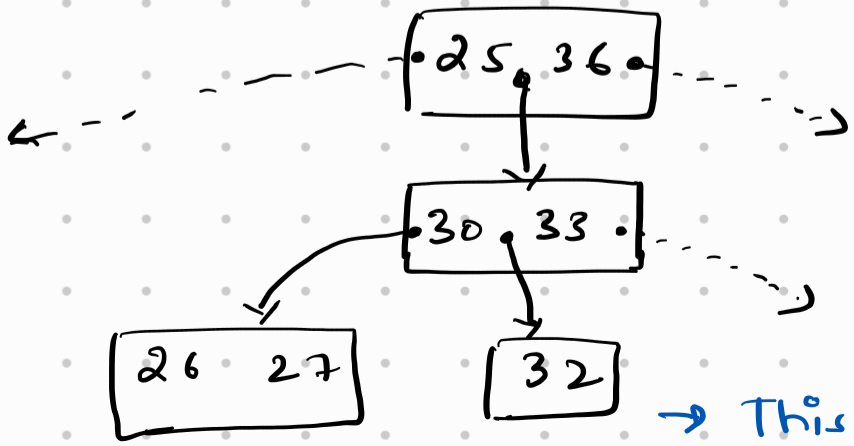
In this case, we need a new separator.

→ Take largest value in the left subtree 27

OR

→ Take smallest value in the right subtree 30

Let's take 30



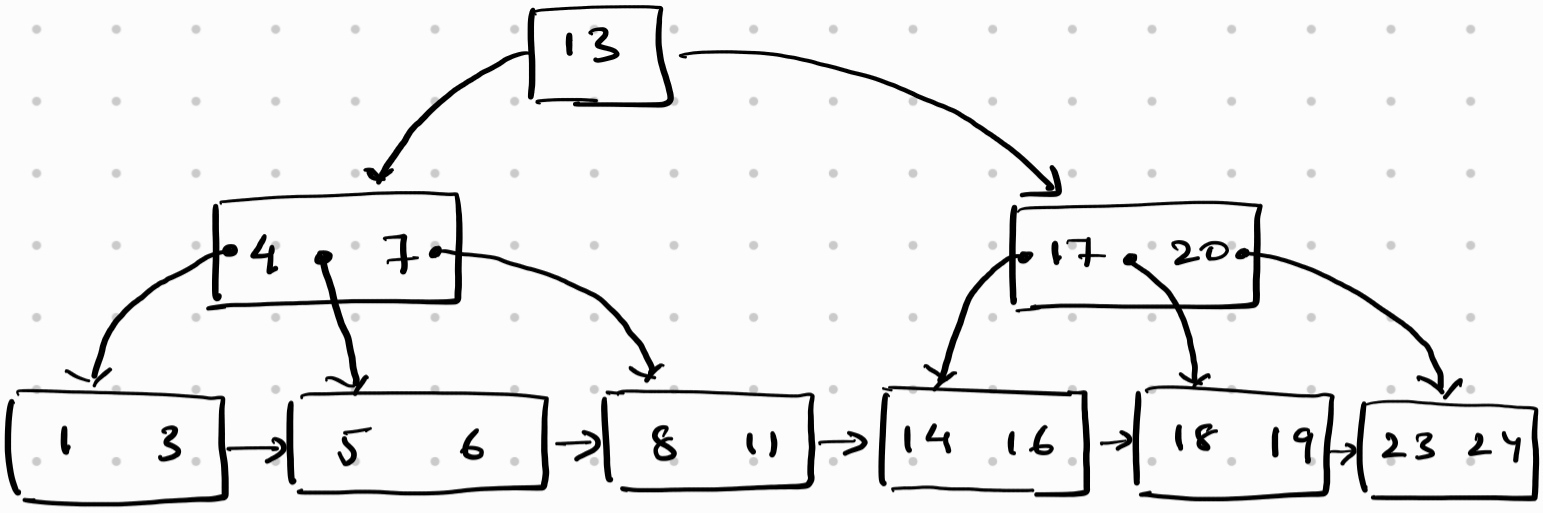
→ This has less than min keys

After we did this notice how | 32 | has only one key, which is less than minimum key.

⇒ we can simply apply merging or take a key from sibling approach.

B^+ Trees

- A B tree node can store Keys and data both. When the Key is found during search operation, we take the data and return the data.
- Usually, in databases, the Key is the primary Key of the table.
- In B^+ trees, the non-leaf nodes only store Keys and child pointers. And, the data is stored only in the leaf nodes. The leaf nodes are linked together like a linked list.
- Since the non-leaf nodes store only Keys and child pointers, that means we can fit more Keys in a single block. (Remember, one node per block) That means, our tree branches out more. That means, the height of the tree is lesser. That means, lesser block reads.
- Since the leaf nodes are linked, so, range query is more efficient. we can directly jump to next block following the link. we just need to reach the first leaf node, then we can easily walk to the next node for sequential access.



- Since the leaf nodes contain the actual data, so, the leaf nodes are used as the main storage structure of the DB and are stored in secondary storage.
- The non-leaf nodes only contain Keys and pointers, so, they are perfect for indexing.
- A B^+ tree can be thought of as multi-level index in which the leaves make up a dense index and the non-leaf nodes make up a sparse index.

Insertion operation in B^+ Trees

-> Insert : 1, 2, 3, 4, 5, 6, 7

-> $m = 4$

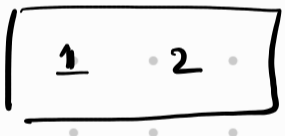
Calculations

1) Maximum no. of Keys per node $= m - 1 = 3$

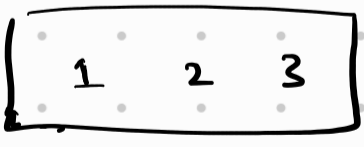
2) Minimum no. of Keys $= \left\lceil \frac{m}{2} \right\rceil - 1 \Rightarrow 1$

Inserting 1

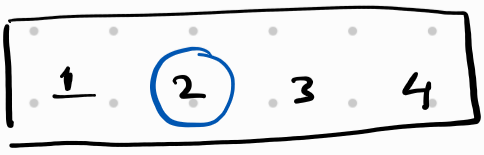
Inserting 2



Inserting 3

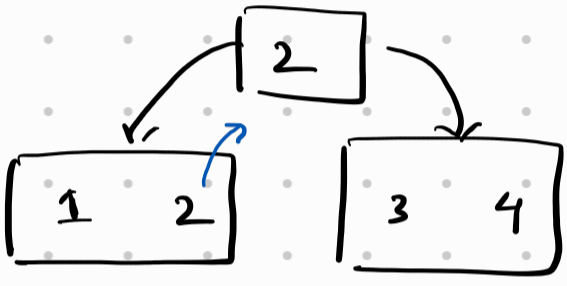


Inserting 4

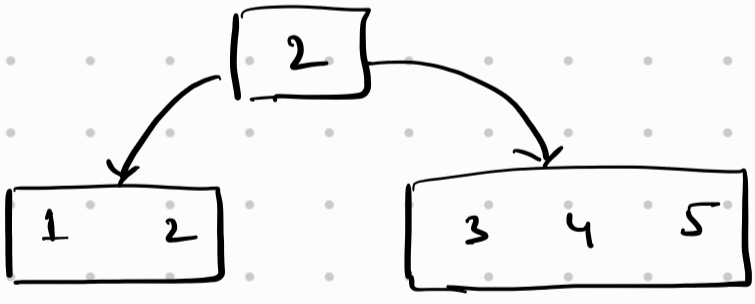


But wait, now our keys are more than max allowed. So, we split. This time let's be right biased, so right nodes will have more keys.

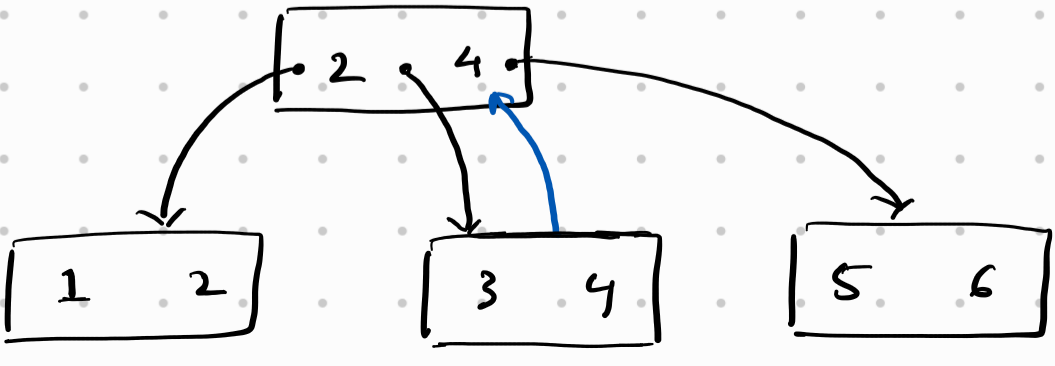
NOTE: Unlike B tree insertion case, here we split the node and "copy" the median upwards if the node is leaf node. Otherwise move the median upwards.



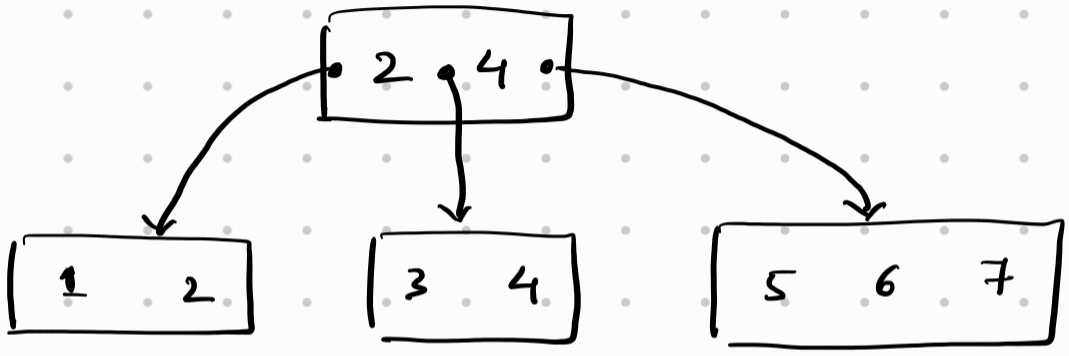
Inserting 5



Inserting 6



Inserting 7



Skipping Deletion case, as we are studing Database indexing for System Design interviews.